Studio Nx

Backend(FastAPI/Python) 25 — SignupRequest · Nickname

2026-10-07

Backend(FastAPI/Python) 25 — 입력 스키마 `SignupRequest` · `Nickname`

항목	내용
파일	`studionx/backend/app/schemas/auth.py` · `studionx/backend/app/api/v1/auth.py`
작업	P1-04 회원가입 — 가입 입력 형식 검사 · PRD 01 FR-101 (P1-04 마지막)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	24 `GET /auth/check-nickname` · 26 `create_access_token()` (P1-05)

1. 하는 일

가입 요청(22)과 닉네임 확인(24)의 **입력 모양**을 정한다. 형식이 틀리면 라우터에 닿기 전에 FastAPI 가 422 로 돌려보낸다.

칸	바꾸기 전 (템플릿)	바꾼 뒤
`email`	`str` (아무 글자)	**`EmailStr`** — 이메일 형식 · 앞뒤 공백 제거 · 254자 이하
`nickname`	`str` 2 ~ 20자 (공백 포함 길이)	**`Nickname`** — 앞뒤 공백을 **먼저 지우고** 2 ~ 20자

입력	바꾸기 전	바꾼 뒤
`nickname: " a "`	통과 (5자) → 저장은 `"a"` (1자)	**422** (공백 빼면 1자)
`nickname: " ab "`	통과 → 서비스가 `"ab"` 로	통과 → 스키마가 `"ab"` 로
`email: "kim@naver"`	통과 → DB 에 이상한 이메일	**422**
`email: " kim@naver.com "`	통과 → 서비스가 공백 제거	통과 → 스키마가 공백 제거

2. 언제 쓰나

쓰는 곳	어떻게
22 `POST /auth/signup`	`data: SignupRequest` — 본문 JSON 을 이 모양으로 검사
24 `GET /auth/check-nickname`	`nickname: Annotated[Nickname, Query()]` — 주소의 `?nickname=` 을 같은 타입으로 검사
23 `GET /auth/check-email`	이미 `EmailStr` 사용 (23 에서 바꿈)

확인 · 가입이 같은 타입을 쓰는 이유: 중복 확인에서 "✓ Available" 이라고 했는데 가입에서 형식 오류가 나면 안 된다. 같은 타입이면 규칙이 한 곳에만 있다.

3. 코드

import (`schemas/auth.py` 맨 위)

```
from datetime import datetime
from typing import Annotated
```

```
from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated` · `EmailStr` · `StringConstraints` 를 이번에 추가했다.

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 조건을 덧붙이는 도구	`Nickname`
`EmailStr`	이메일 형식 타입 (23 에서 설치한 `email-validator` 가 검사)	`email`
`StringConstraints`	글자 조건 묶음 (공백 제거 · 길이)	`Nickname`

함수 (`schemas/auth.py`)

```
NICKNAME_MIN, NICKNAME_MAX = 2, 20

# 닉네임 입력 타입 — 앞뒤 공백을 먼저 지우고 길이를 잼 (가입 · 중복 확인이 함께 씀, 2026-10-07)
Nickname = Annotated[
    str,
    StringConstraints(strip_whitespace=True, min_length=NICKNAME_MIN, max_length=NICKNAME_MAX),
]


class SignupRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    nickname: Nickname
    password: str = Field(min_length=1, max_length=100)  # 정확한 8~20자 검사는 check_password_rules
    privacy_agreed: bool
    privacy_version: str = Field(max_length=20)
```

(`api/v1/auth.py` 의 24 라우터)

```
    nickname: Annotated[Nickname, Query()],     # 바꾸기 전: Annotated[str, Query(min_length=…, max_lengt
h=…)]
```

24 에서 넣었던 `min_length` · `max_length` 는 `Nickname` 안으로 들어가서, 라우터의 `NICKNAME_MIN` · `NICKNAME_MAX` import 는 지웠다.

문법 풀이

import 문법	뜻
`from pydantic import BaseModel, EmailStr, Field, StringConstraints`	pydantic 에서 네 이름을 한 줄로 가져옴

식	뜻
`Nickname = Annotated[str, StringConstraints(…)]`	**타입에 이름을 붙임** — "조건이 붙은 글자" 를 `Nickname` 한 단어로 여러 곳에서 씀
`strip_whitespace=True`	검사 **전에** 앞뒤 공백을 지움. 지운 값이 그대로 `data.nickname` 에 들어감
`min_length` · `max_length`	공백을 지운 **뒤** 길이
`EmailStr = Field(max_length=255)`	이메일 형식 + 255자 이하. 실제로는 이메일 규격 상한 254자에서 먼저 막힘
`Annotated[Nickname, Query()]`	같은 타입을 주소 값(`?nickname=`)에도 씀

4. 흐름 (요청이 들어올 때)

1. FastAPI 가 JSON · 주소 값을 꺼낸다.
2. `email` → 앞뒤 공백 제거 → 형식 검사. `nickname` → 앞뒤 공백 제거 → 2 ~ 20자 검사.
3. 하나라도 틀리면 02 처리기가 422 + `details` (어느 칸인지).
4. 모두 맞으면 정리된 값(공백 지운 이메일 · 닉네임)으로 라우터 → 서비스(21).

5. 왜 이렇게

선택	이유
형식 검사는 스키마(입구)에서	틀린 입력이 서비스 · DB 까지 가지 않는다. 오류 모양도 다른 입력 오류와 같다(02)
비밀번호는 스키마에서 1 ~ 100자만	8 ~ 20자 · 조건 검사는 14 가 한다. 조건별 실패 목록(`["upper", …]`)을 화면에 돌려주려면 서비스에서 해야 한다
이메일 소문자는 스키마가 아니라 17 에서	로그인(P1-05) · 중복 확인도 17 을 거친다. 비교 기준을 한 함수에 모은다
서비스(21)의 `strip()` · `normalize_email()` 은 그대로 둠	스키마를 거치지 않고 서비스를 부르는 곳(시험 · 나중의 관리 기능)이 있어도 같은 기준이 지켜진다
`Nickname` 을 `schemas/auth.py` 에	P1-08 회원 정보 수정(`schemas/user.py`)도 닉네임을 바꾼다. 그때 이 타입을 가져다 쓰면 된다

6. 시험

`tests/test_auth_schemas.py` (새로 만듦 · 16개)

시험	확인한 것
닉네임 통과 4개	`beatmaker` · `" beatmaker "` → `beatmaker` · `" ab "` → `ab` · 20자
닉네임 실패 5개	1자 · `" a "` · 공백만 · 21자 · 공백 빼도 21자 → `nickname` 오류
이메일 공백	`" kim@naver.com "` → `kim@naver.com`
이메일 실패 5개	`abc` · `a b@x.com` · `kim@naver` · `deleted+85@withdrawn.invalid` · 빈 글자 → `email` 오류
이메일 길이	254자는 통과, 300자 넘는 것은 `email` 오류

`tests/test_auth_routes.py` 에 3개 추가 (모두 27개)

시험	확인한 것
`GET /check-nickname?nickname= a`	422 (24 에서 남았던 차이가 고쳐짐)
`POST /signup` 닉네임 `" a "`	422, `details` 에 `nickname`
`POST /signup` 이메일 `kim@naver`	422, `details` 에 `email`

전체 시험 186개 통과, `ruff` 검사 통과.